

מבחן במבוא למדעי המחשב – 67101

מועד א' 30.1.17

מורי הקורס: אביב זהר ונעם ניסן

במבחן זה שני חלקים. יש לענות על כל 10 השאלות בחלק א', ולבחור 4 מתוך 6 השאלות בחלק ב' (אין לענות על יותר מ-4 שאלות בחלק ב').

יש לענות על כל השאלות בגוף השאלון – בתוך "קופסאות התשובה" בלבד. אין להגיש דפי טיוטא – הם לא יבדקו.

חומר מותר לשימוש בבחינה: אין.

משך הבחינה: 3 שעות.

סמנו כאן על אילו שאלות בחרתם לענות בחלק ב :

6	5	4	3	2	1
---	---	---	---	---	---

(הקיפו בעיגול)

חלק א' – שאלות סגורות (40% -- יש לענות על כל 10 הסעיפים, כל סעיף מזכה ב-4 נק')

בחלק זה של המבחן יש לכתוב את התשובה הסופית בלבד בתוך המשבצת המתאימה. אין צורך לנמק, ונימוקים גם לא יקראו.

1. מה ידפיס הקוד הבא?

```
g = lambda x: x+7
foo = lambda f: (lambda x: f(x+1)*2)

print( g(3), (foo(g))(3), (foo(foo(g)))(3) )
```

2. נתונה פונקציה $\text{do_log_work}(v, L)$ שמקבלת ערך v ורשימה L כקלט וזמן הריצה שלה הינו $O(\log N)$ כאשר N הוא אורך הרשימה L . מה סיבוכיות זמן הריצה של הפונקציה הבאה?

```
def foo(L):
    for v in L:
        do_log_work(v, L)
```

זמן הריצה של הפונקציה (כתלות באורך N של הרשימה L) הוא:

3. מה ידפיס הקוד הבא?

```
def foo(a):
    c = a
    c[0] = 3
    print(c)

g = [1, 2, 3]
foo(g)
print(g)
```

4. מה ידפיס הקוד הבא?

```
lst = [1,2,3,4]
x = iter(lst)
y = iter(lst)
z = x
print(next(x), next(y), next(z))
print(next(x), next(y), next(z))
```

5. מה ידפיס הקוד הבא?

```
def foo(n):
    if n <= 0:
        return tuple()
    tup = foo(n - 1)[::-1]
    return tup + (n,)

print(foo(5))
```

6. מה ידפיס הקוד הבא?

```
letters = ["a","b","c"]
text = "aabbcc"
my_dict = {letters[i]: i*3 for i in range(len(letters))}
new_list = [my_dict[letter] for letter in text]
print(new_list)
```

7. מה ידפיס הקוד הבא?

```
numbers = ["2","1","0","a"]
try:
    for number in numbers:
        try:
            print(6 // int(number), end = " ")
        except(ZeroDivisionError): print("oops", end = " ")
        finally: print("ok", end = " ")
except: print("Boom")
```

8. אלו מהטענות הבאות נכונות (כאשר F הינה פונקציה ממחרזות למחרזות)?

- (א) לכל תוכנית פייתון שמחשבת פונקציה F יש מכונת טיורינג שמחשבת את אותה פונקציה
- (ב) לכל מכונת טיורינג שמחשבת פונקציה F יש תוכנית פייתון שמחשבת את אותה פונקציה
- (ג) לכל פונקציה F קיימת מכונת טיורינג שמקבלת כקלט מחרוזת X ומחשבת את $F(X)$
- (ד) ישנה מכונת טיורינג המחשבת את $F(X)$, כאשר נותנים לה כקלט מחרוזת X ומחרוזת המייצגת תכנית פייתון עבור פונקציה F .

9. מהי סיבוכיות זמן הריצה של התוכנית הבאה כפונקציה של N ?

```
import functools
num_list = list(range(N))
for i in range(1, len(num_list)+1):
    shorter_list = num_list[:i]
    y = functools.reduce(lambda u,v: u+v, shorter_list)
    print(y)
```

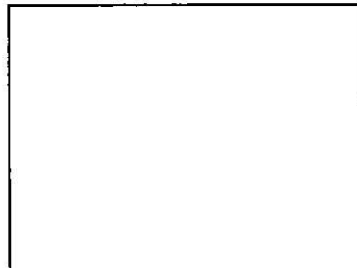


10. מה ידפיס הקוד הבא?

```
class A:
    def __init__(self):
        self.x = 1
        A.y = 10
        print(self.x + A.y)

    def f(self):
        self.x = self.x + 1
        A.y = A.y + 10
        return self.x + A.y

a=A()
b=A()
print(a.f())
print(b.f())
print(a.f())
```



חלק ב' – שאלות תכנות (60% -- יש לענות רק על 4 מתוך 6 השאלות, כל שאלה 15 נק')

יש לענות על כל שאלה רק בתוך הקופסה שניתנה בדף. מומלץ להשתמש במחברת הטיוטא כדי לגבש את הפתרון ורק כשהוא מושלם להעתיקו לטופס המבחן. אין צורך בהערות או בתיעוד. יש לכתוב קוד בהיר ויעיל.

1. כתבו פונקציה `all_increasing(lst)` המקבלת רשימה של מספרים שונים זה מזה ומדפיסה את כל תתי סדרות המספרים העולות של הרשימה המקורית.

```
all_increasing([1,4,3,2]) :אמת
```

```
[1] [2] [3] [4] [1] [1, 2] [1, 3] [1, 4]
```

(או כל סדר אחר של אותן רשימות)

במילים אחרות, תודפס כל רשימה שניתן לקבל ע"י מחיקת איברים בלבד (ללא שינוי סדר) מהרשימה המקורית שנותנת סדרת המספרים. (גם רשימה באורך 0 וגם רשימה באורך 1 נחשבות רשימות עולות).

```
def all_increasing(lst):
```

2. נתונה מחלקה עבור איבר (Node) של רשימה מקושרת:

```
class Node:
    def __init__(self, data, next=None):
        self.data = data
        self.next = next
```

כמו כן נתונה מחלקה עבור רשימה מקושרת (linked list):

```
class LinkedList:
    def __init__(self):
        self.__head = None
        self.__tail = None
```

ממשו את המתודה (פונקציית מחלקה) `is_repetative(self, lst)` (של המחלקה `LinkedList`) אשר מחזירה ערך `True` אם המידע (`data`) שבאיברי הרשימה המקושרת הוא בדיוק מספר שלם של חזרות של הרשימה `lst` ו-`False` אחרת. `lst` היא רשימת פייתון רגילה, לא רשימה מקושרת. ניתן להניח שהרשימה המקושרת וגם `lst` מכילות לפחות איבר אחד כל אחת.

דוגמא: אם `myList` (אובייקט מטיפוס `LinkedList`) מכיל את הרשימה `[a→b→a→b→a→b]` אזי:

- `myList.is_repetative(["a", "b"])` True יחזיר
- `myList.is_repetative(["a", "b", "a"])` False יחזיר
- `myList.is_repetative(["a", "b", "a", "b"])` False יחזיר
- `myList.is_repetative(["c"])` False יחזיר

```
def is_repetitive(self, lst):
```

3. נתונה רשימה 1st אשר ממוינת מתחילתה ועד אינדקס i , וגם מאינדקס $i+1$ עד סוף הרשימה, אך כל האיברים שבין אינדקס $i+1$ ועד סוף הרשימה קטנים מאלו שנמצאים בתת הרשימה מאינדקס 0 עד אינדקס i . כלומר מתקיים:

$$\text{lst}[i+1] < \text{lst}[i+2] < \dots < \text{lst}[n-1] \text{ DAI } \text{lst}[0] < \text{lst}[1] < \dots < \text{lst}[i] \\ \text{lst}[n-1] < \text{lst}[0] \text{ DAI}$$

כיתבו פונקציה `find_discontinuity(lst)` המחזירה את האינדקס `i` המקיים את התנאים הללו.

הערה: ניקוד מלא דורש זמן ריצה של $O(\log N)$ כאשר N הוא אורך הרשימה.

```
def find_discontinuity(lst):
```


4. כתבו פונקציה `inverse` המקבלת כקלט מספר טבעי N ופונקציה f כאשר מובטח כי $f: \{0, 1, \dots, N-1\} \rightarrow \{0, 1, \dots, N-1\}$ הינה חד-חד ערכית, ומחזירה את הפונקציה ההופכית של f .

לדוגמא:

```
def f(x): return (x+1)%7  
g = inverse(f, 7)  
print(g(3))
```

ידפיס "2".

```
def inverse(f, N):
```

5. ממשו מחלקה Range2D המייצגת טווח של נקודות בדו-מימד עם קואורדינטות שלמות, כאשר טווח ה- x מתחיל ב-0 וממשיך עד לגבול העליון הנתון (לא כולל), וכנ"ל לגבי טווח ה- y . עליכם לתמוך בפעולות הבאות:

א. יצירת אובייקט:

```
rect1 = Range2D(3,2)
```

יוצר אובייקט Range2D עם קואורדינטות x בערכים 0,1,2 וקואורדינטות y בערכים 0,1

ב. איטרטור: על אובייקט ה-Range2D ניתן לעבור בעזרת לולאה באופן הבא:

```
for point in rect1: #iterate over all points in the Range2D
    print(point, end=" ")
```

ידיפים: $(0,0)$ $(0,1)$ $(1,0)$ $(1,1)$ $(2,0)$ $(2,1)$ כאשר כל נקודה בטווח הינה זוג (x,y)

ג. בדיקת הכלה: יש גם לממש מתודת contains שבודקת אם טווח אחד מוכל בטווח אחר:

```
print(rect1.contains(Range2D(2,2))) # True ט'פ'ט
print(rect1.contains(Range2D(2,4))) # False ט'פ'ט
print(rect1.contains(Range2D(5,5))) # False ט'פ'ט
print(rect1.contains(rect1)) # True ט'פ'ט
```

```
class Range2D:
```

6. כתבו פונקציה `most_common(1st)` המקבלת רשימה של מחרוזות ומחזירה את האיבר הכי נפוץ ברשימה. רק פתרון שעובד בזמן $O(n)$ יקבל ניקוד מלא (רמז: ניתן להשתמש במילון). ניתן להניח שברשימה איבר אחד לפחות. במידה ויש כמה איברים נפוצים ביותר מותר לפונקציה להחזיר כל אחד מהם.

דוגמא: עבור הרשימה ["alice", "bob", "alice", "carol", "carol", "alice"] יוחזר הערך "alice".

```
def most_common(lst):
```

בהצלחה!!!